```
create view all-customer as
    (select branch-name, customer-name
     from depositor, account
     where depositor.account-number = account.account-number)
   union
    (select branch-name, customer-name
     from borrower, loan
     where borrower.loan-number = loan.loan-number)
```

The attribute names of a view can be specified explicitly as follows:

```
create view branch-total-loan(branch-name, total-loan) as
select branch-name, sum(amount)
from loan
groupby branch-name
```

The preceding view gives for each branch the sum of the amounts of all the loans at the branch. Since the expression **sum**(*amount*) does not have a name, the attribute name is specified explicitly in the view definition.

View names may appear in any place that a relation name may appear. Using the view *all-customer*, we can find all customers of the Perryridge branch by writing

```
select customer-name
from all-customer
where branch-name = 'Perryridge'
```

4.8 Complex Queries

Complex queries are often hard or impossible to write as a single SQL block or a union/intersection/difference of SQL blocks. (An SQL block consists of a single **select from where** statement, possibly with **groupby** and **having** clauses.) We study here two ways of composing multiple SQL blocks to express a complex query: derived relations and the **with** clause.

4.8.1 Derived Relations

SQL allows a subquery expression to be used in the **from** clause. If we use such an expression, then we must give the result relation a name, and we can rename the attributes. We do this renaming by using the **as** clause. For example, consider the subquery

```
(select branch-name, avg (balance)
 from account
 group by branch-name)
as result (branch-name, avg-balance)
```

This subquery generates a relation consisting of the names of all branches and their corresponding average account balances. The subquery result is named *result*, with the attributes *branch-name* and *avg-balance*.

To illustrate the use of a subquery expression in the **from** clause, consider the query "Find the average account balance of those branches where the average account balance is greater than $1200." We wrote this query in Section 4.4 by using the **having** clause. We can now rewrite this query, without using the **having** clause, as follows:

```
select  branch-name, avg-balance
from (select branch-name, avg (balance)
        from account
        group by branch-name)
      as branch-avg (branch-name, avg-balance)
where avg-balance > 1200
```

Note that we do not need to use the **having** clause, since the subquery in the **from** clause computes the average balance, and its result is named as *branch-avg*; we can use the attributes of *branch-avg* directly in the **where** clause.

As another example, suppose we wish to find the maximum across all branches of the total balance at each branch. The **having** clause does not help us in this task, but we can write this query easily by using a subquery in the **from** clause, as follows:

```
select max(tot-balance)
from (select branch-name, sum(balance)
      from account
      group by branch-name) as branch-total (branch-name, tot-balance)
```

4.8.2 The with Clause

Complex queries are much easier to write and to understand if we structure them by breaking them into smaller views that we then combine, just as we structure programs by breaking their task into procedures. However, unlike a procedure definition, a **create view** clause creates a view definition in the database, and the view definition stays in the database until a command **drop view** *view-name* is executed.

The **with** clause provides a way of defining a temporary view whose definition is available only to the query in which the **with** clause occurs. Consider the following query, which selects accounts with the maximum balance; if there are many accounts with the same maximum balance, all of them are selected.

```
with max-balance (value) as
     select max(balance)
     from account
select account-number
from account, max-balance
where account.balance = max-balance.value
```